

# COMPARATIVE REWARD ENGINEERING FOR SPECIALIZED RL AGENTS IN ROCKET LEAGUE

**By: Samiul Saimon**

YOU: What a save!  
YOU: What a save!  
YOU: What a save!  
Chat disabled for 3 seconds.

## WHAT IS ROCKET LEAGUE?

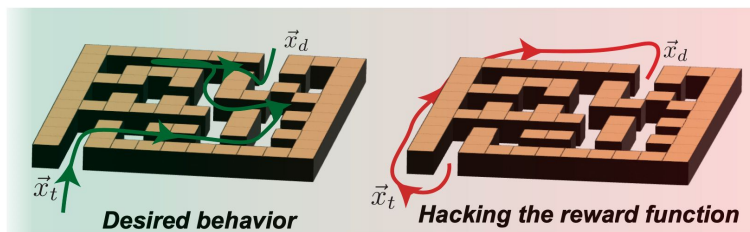


*Cars + Soccer*



# PROJECT BREAKDOWN

- Focusing on a specialized mechanic to keep a reasonable scope: Training Rocket League Agents **Defensive behavior/plays**
- Comparing how **different reward designs** affect how quickly and well they learn under identical PPO and self-play conditions
- **Reward Hacking** case and **iterated correction** results



$$r(s_t, a_t) = -\|\vec{x}_t - \vec{x}_d\|^2$$

(Reward is a form of “Minimize distance to goal”)

## REWARD DESIGN AND SHAPING



### REWARD DESIGN

Crafting the reward signal an RL agent receives



### REWARD SHAPING

Modifying the reward signal to improve learning



### COMMON REWARD DESIGN TECHNIQUES

Dense vs. sparse rewards  
• Shaping terms • Penalties  
• Curriculum rewards



### CHALLENGES AND PITFALLS

Reward hacking • Over-shaping  
Delayed rewards • Ambiguity



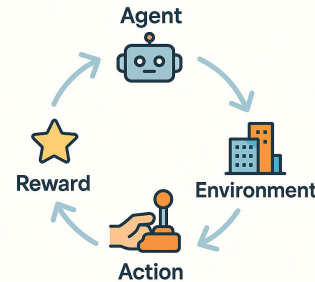
# DEEP REINFORCEMENT LEARNING

DRL combines **reinforcement learning** (trial-and-error, reward-based learning) with **deep neural networks**, enabling agents to make optimal decisions from complex, high-dimensional data by approximating solutions.

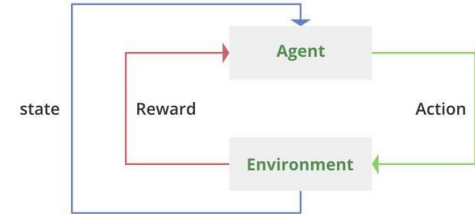
## Four Core elements of RL:

- **Agents:** the cars playing the game
- **Environment:** the stadium, the ball, the boost pads, other players, etc.
- **Actions:** where to move, whether to boost, whether to hit the ball, jump
- **Rewards:** making successful defensive plays (defined by multiple reward models)

The Core Feedback Loop of Reinforcement Learning



Created by Mehul Ligade · x.com/MehulLigade





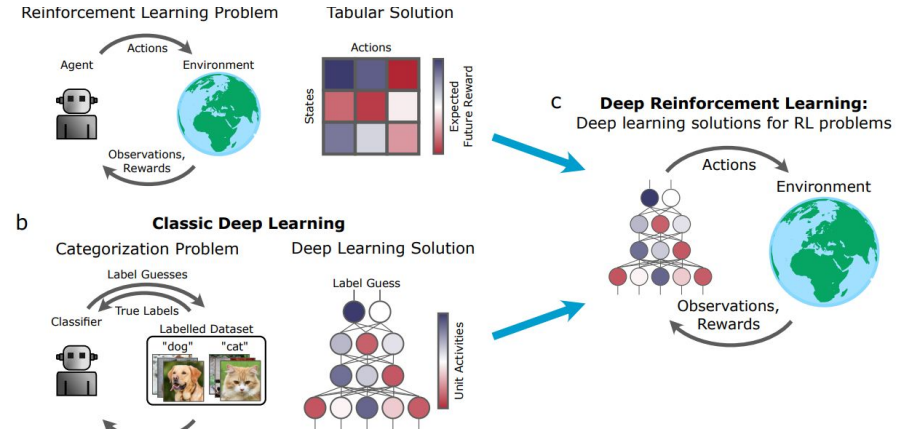
# DEEP REINFORCEMENT LEARNING

DRL combines **reinforcement learning** (trial-and-error, reward-based learning) with **deep neural networks**, enabling agents to make optimal decisions from complex, high-dimensional data by approximating solutions.

## Deep Neural Networks:

- act as "function approximators"
- allow the algorithm to analyze large messy datasets to estimate the value of different actions.

**Advantage:** allow the system to generalize and learn patterns rather than having to map every possible situation.





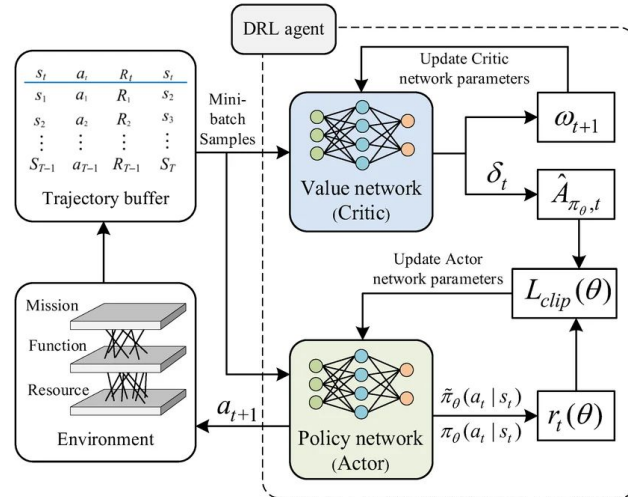
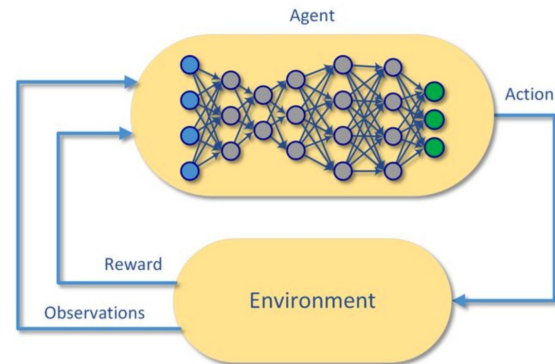
# RL ALGORITHM: PROXIMAL POLICY OPTIMIZATION (PPO)

- Forces the agent to make **small, safe incremental changes**. Keeps the new, updated strategy close to the old, working strategy (hence proximal). To avoid performance collapse - this is more stable.

Actor-Critic (Hybrid) RL Method: It uses two neural networks:

- The Actor**: Decides which move to make (the **Policy**).
- The Critic**: Watches the Actor and predicts how much reward that move will actually get (the Value).

The PPO algorithm is the "Coach" that trains the Actor-Critic "Brain" inside the Agent "Car."



# MACHINE SPECS

- **Intel i5-11500 (6 cores / 12 threads)** - Parallel RocketSim workers do most of their work on CPU; 12 logical processors allows us to run 8+ game simulations at once, keeping the GPU constantly busy with new data.
- **16 GB RAM:** High-speed memory that ensures the physics engine and the AI trainer can swap data without lag.
- **x64 Windows:** Provides full compatibility with the specific drivers and software binaries used in this project.
- **NVIDIA RTX 3060 (12GB VRAM):** CUDA accelerates PPO updates. Like the "math muscle." Its massive memory means we can increase training complexity (batch/workers) without running out of space.



# TECH STACK



- **Python 3.9:** Stable language version for the RL libraries this project pins to.
- **Miniconda:** Used for environment management to handle Python 3.9 and C++ dependencies/CUDA-enabled PyTorch cleanly.
- **PyTorch (CUDA 12.1 Build):** speeds up training by offloading heavy mathematical computations from a computer's CPU to its GPU, instead of only CPU.
- **RLGym v2 (rlgym + rlgym-rocket-league):** the "brain" or the framework that handles the reinforcement learning logic (rewards, terminal conditions, agent training). And the environment implementation.
- **RocketSim:** Fast stripped-down physics engine so training can take millions of steps without running the real game.
- **RLGym-PP0:** high-throughput training framework that acts as the 'coach' for the agent. Manages entire learning loop, handling parallel environment workers, checkpointing, and metric tracking - to turn raw experience into a skilled Rocket League agent.
- **NumPy:** Vector math for rewards/metrics (positions, distances, normalization).
- **Weights & Biases (wandb):** A dashboard to track training progress and compare different reward types metrics.
- **RLViser:** visualizer for local 3D playback of a saved checkpoint (watching agent after training).
- **Windows 11 + Powershell:** The native environment where all our tools and commands are tested.



# REWARD OVERVIEW

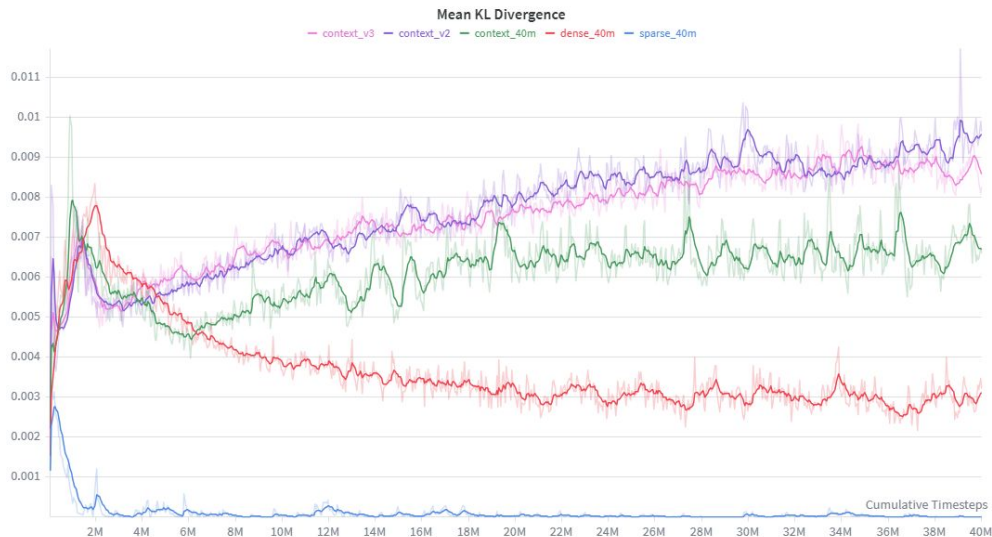
| Reward Type      | Description                                                                                                                     |
|------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Sparse           | only reward a defensive touch on agents own half to encourage staying in the goal area.                                         |
| Dense            | based on how much the ball's velocity points away from the agent's own goal (encourages clearing the ball out of the goal area) |
| Context-Aware    | dense physics signal plus positioning, orientation towards ball, and boost context (for better reactions).                      |
| Context-Aware v2 | V1 + Penalty for drifting far from goal, prolonged no touch behavior, center ball unchallenged (stop reward hacking).           |
| Context-Aware v3 | V2 + event-based positive reward on any ball contact (encourage hitting ball).                                                  |

GRAPHS



# STABILITY/OPTIMIZATION HEALTH

Average difference between old and new agent policy (how fast its learning)

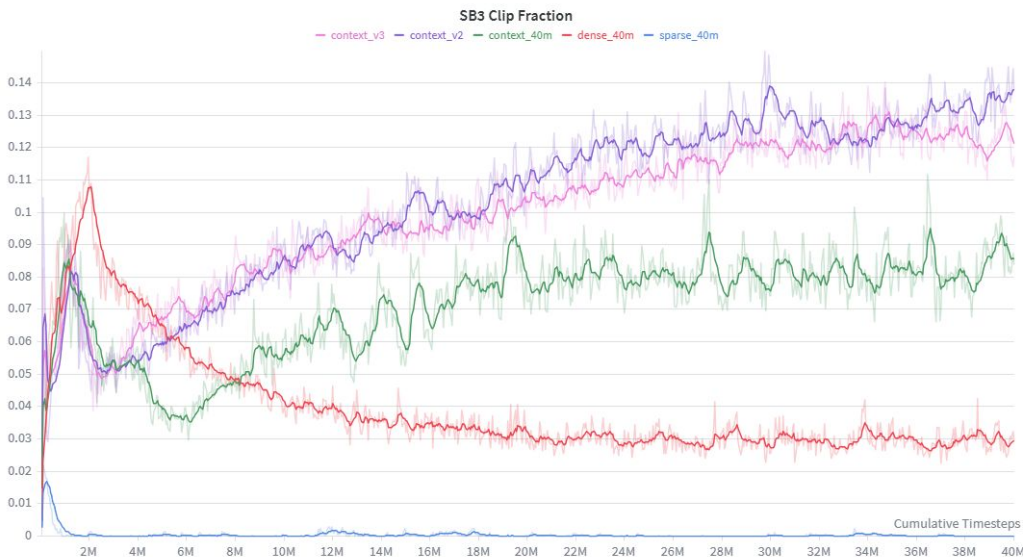


- Sparse essentially stopped learning (flatlined)
- Dense is healthy (learned quickly then gradually refined policy)
- Context runs very aggressive (more adventurous over time)

# STABILITY/OPTIMIZATION HEALTH



How much the agent is trying to change in one go (and being stopped)

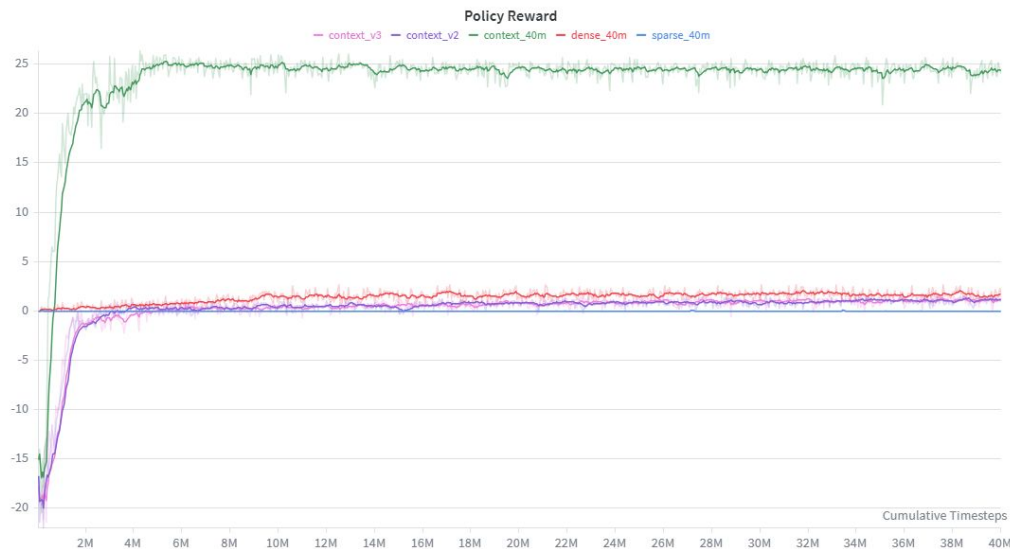


- Context runs hitting 0.12 to 0.14 (high)
- Dense spiked early than calmed down
- Sparse effectively zero (isn't even trying)

# POLICY REWARD



Scorecard, can't be compared across different reward strategies



- Context\_40m a **hacked policy**, v2 and v3 correct that exploit. V3 better than v2 so adding touch bonus helped.
- **Dense is healthy** (steady classic climb)
- **Sparse near zero** (practically not learning)

# CONTEXT\_40M HAD A HIGHER POLICY REWARD THAN CONTEXT\_V2 - ISN'T THAT BETTER?

Policy Reward is only meaningful within one reward function. It is not comparable across different reward designs because they're on different scales / measure different things. A higher number does not mean the agent is a better goalie - it just means the agent extracted more of that specific reward formula.

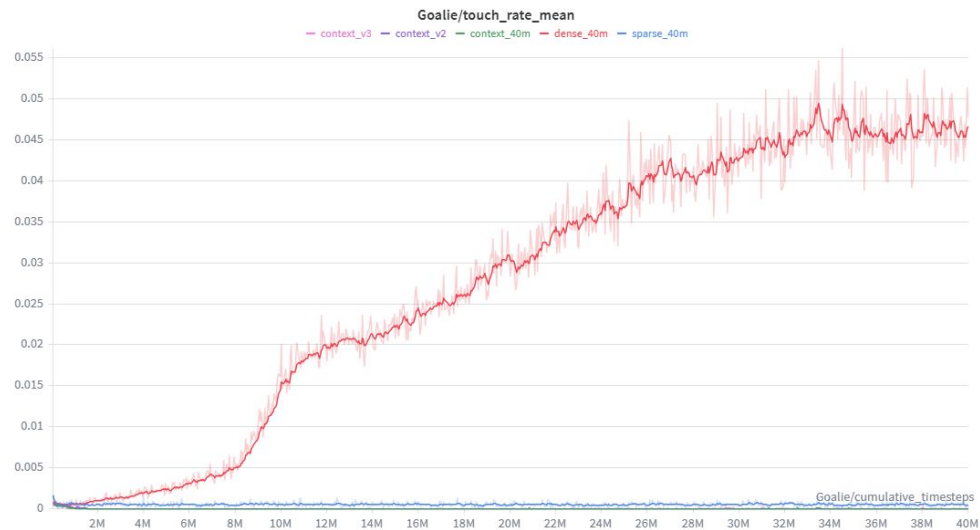
- context\_40m got **Policy Reward  $\approx 24$**  but had **touch\_rate = 0**, **ball\_speed = 0**, was glued near goal with high passive boost. That is a textbook **reward-hacked policy**: **it found that staying near net + facing the ball + holding boost above 20 gives a constant positive sum**, and **avoided the volatile dense term entirely (basically camping)**.
- context\_v2 got **Policy Reward  $\approx 0.95$**  because we removed those easy bonuses and added no-touch penalties. The **exploit is gone** (good), and **positioning is healthier** (dist\_to\_own\_goal 1624  $\rightarrow$  2607 (less glued to net), dist\_to\_ball 3634  $\rightarrow$  2596 (closer to ball), boost 22.7  $\rightarrow$  8.9 (less passive boost farming)).

So: context\_v2 is better than context\_40m because the reward it gets actually reflects goalie behavior, even though the number is smaller. Higher reward is only better if the reward function is actually aligned with the task.

Stability indicators getting more aggressive: prioritizing quicker learning speed over safety constraints. Taking higher risks.

# TOUCH RATE MEAN

How often agent makes contact with ball

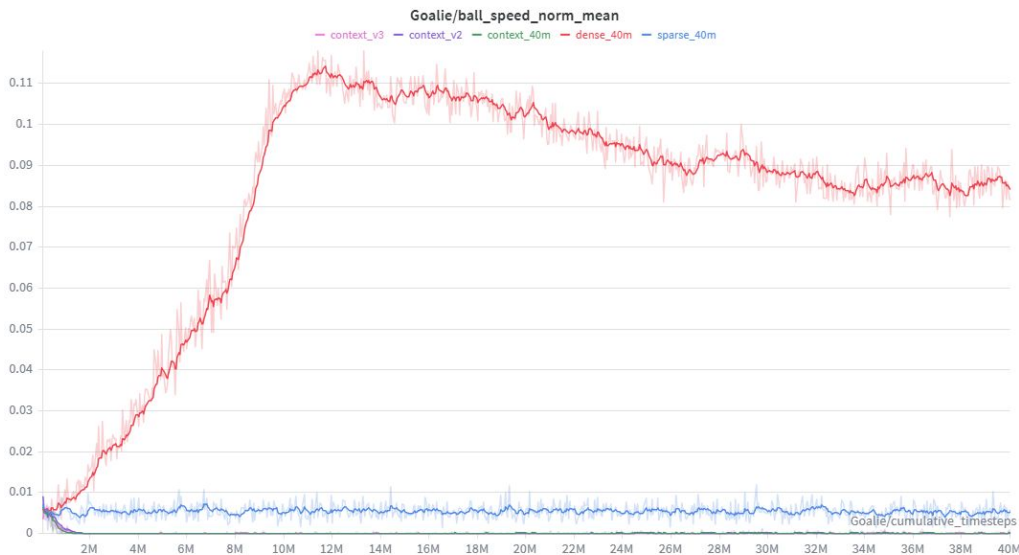


- Context\_40m had high policy but near zero touch rate **confirming exploit**. v2,v3 although fixing exploit, still near zero touch rate.
- Dense is healthy (steady classic climb), touching ball more and more.
- Sparse near zero like context (practically not hitting the ball)



# BALL SPEED NORMALIZED MEAN

how much the ball is actually moving during the episode



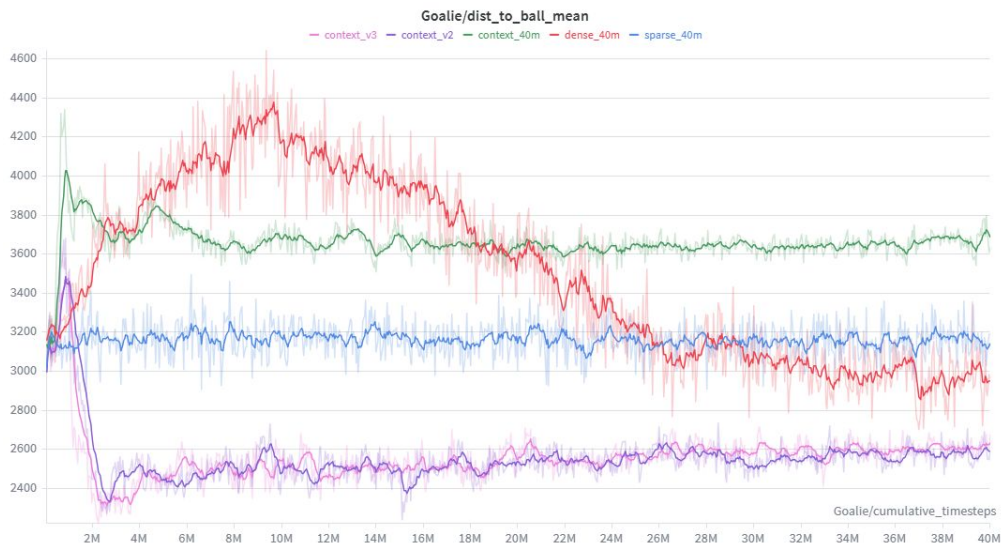
- Dense most effective (ball has high kinetic energy, actually moving)
- All context runs near 0 (40m exploiting, v2/v3 watching stationary ball)
- Sparse has tiny constant noise (negligible) (barely moving ball)





# DISTANCE TO BALL MEAN

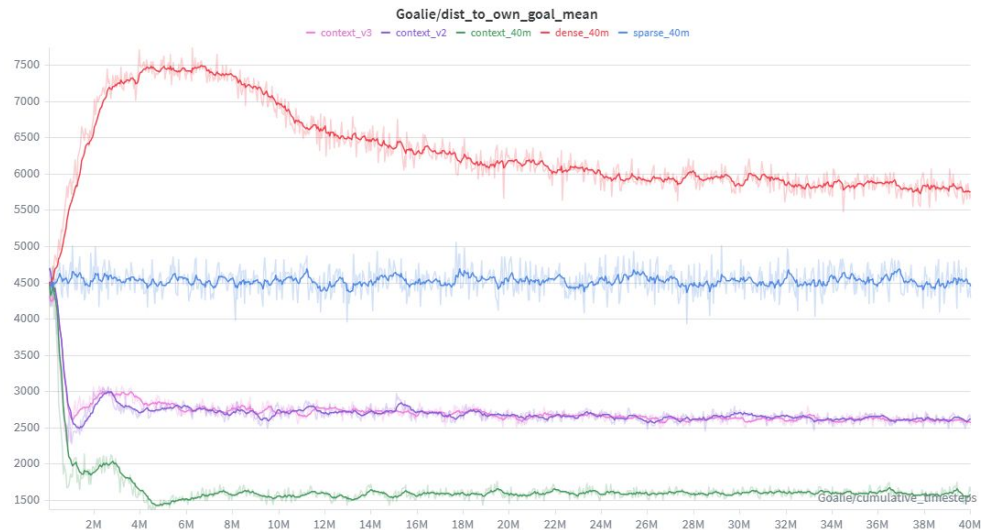
Ball chaser or camper?



- V2 and v3 crashed early to the lowest distance (~2500) and stayed there. Not touching but shadowing ball effectively.
- Dense started far away (up to 4400), then slowly migrated toward the ball, ending around 2900. (healthy)
- Sparse and context\_40m camping (exploit and no incentive).



# DISTANCE TO OWN GOAL MEAN



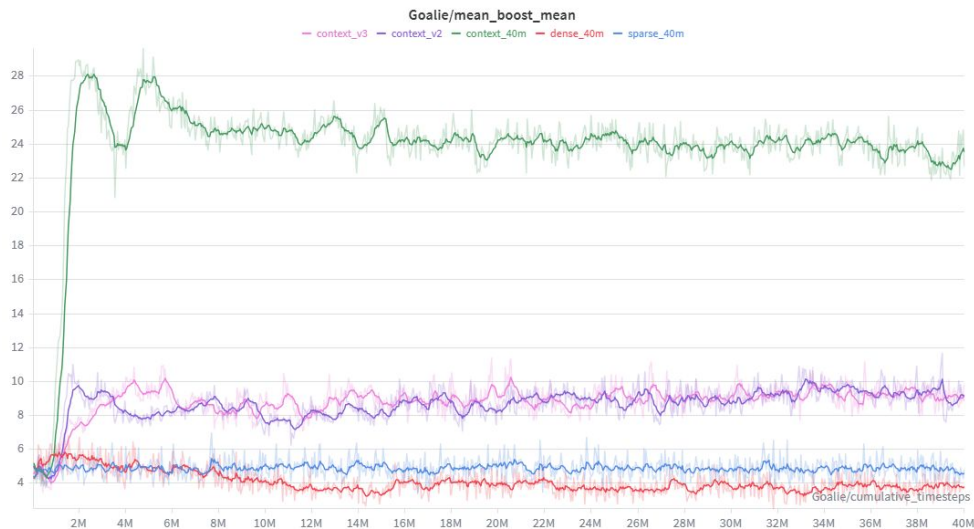
Higher: Too aggressive and far out or  
Lower: too caved in and camping?

- Context\_40m line plummeted to the absolute bottom (~1500) and stayed locked there. (exploit)
- V2, v3 higher than context\_40m but still relatively low. Fixed the exploit, patrolling the area.
- Dense roaming extremely far from the net. great ball-striker (high touch rate), but a potentially "reckless" goalie.
- Sparse flat at 4500 (starting point)



# MEAN BOOST

Amount of boost agent is holding



- Sparse and dense have no boost incentive so lowest.
- Context\_40m consistently above 20, earning passive points.
- Context v2 and v3 broke the addiction and focusing on defense near ball.



# CONTEXT\_V2 BREAKDOWN

## **“Do nothing and win”**

- Suppressed the exploits but only added negative signals, essentially more punishment than reward.
- agent learned “don’t do the thing that loses points,” but never got a strong positive pull toward actually engaging the ball.
- Aggressive learning: Since the most recent thing it learned was "getting close to the ball is bad," it pivoted its strategy very fast and very hard toward staying away.

No risks, low engagement policy, so its not rly playing the game.

It’s like Romans V Carthage:

- Rome was famous for its system of military decorations creating a “high-engagement policy” and “strong positive pull”.
- Carthage was notoriously ruthless with its generals creating an environment of extreme fear (low-engagement policy).



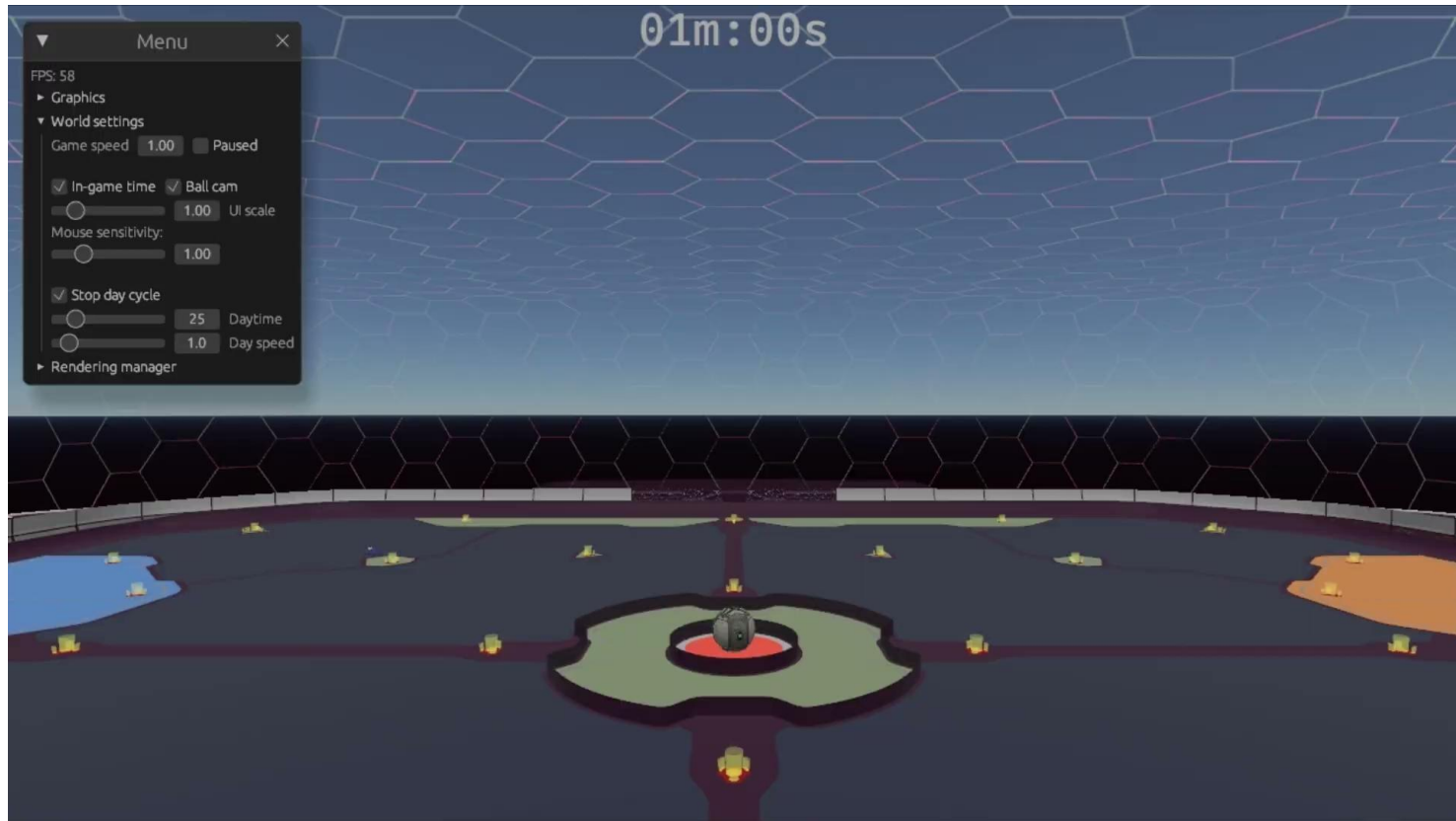
## RESULTS:



| Reward Type                   | Data                                                                                                           | Findings                                                                                                                                                                   |
|-------------------------------|----------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| sparse_40m                    | Near zero ball touches and ball speed.                                                                         | Stagnant/weakest learning, needs more incentive.                                                                                                                           |
| dense_40m                     | Goalie/ numbers* show real ball interaction                                                                    | High engagement and ball speed, but prone to leaving the net wide open due to aggression and over-forward tendency.                                                        |
| Context_40m                   | highest Mean Boost (~24) and lowest Dist to Own Goal                                                           | Camping + Farming: It spent the episodes zipping between boost pads and the net to collect passive points.                                                                 |
| Context_v2 (more penalties)   | Shows one of the lowest Distance to Ball (crashing to ~2500)                                                   | Good positioning: passively shadowing the ball. Reward for "being near the ball" is currently stronger than the reward for "moving the ball."                              |
| Context_v3 (ball touch bonus) | one of the lowest Distance to Ball (crashing to ~2500), more jitter in KL, and slightly higher rewards than v2 | Positioning + Hitting: the Touch Bonus is working. It is forcing the agent to break its "shadowing" habit to get occasional hits and actually play (hasn't fully mastered) |

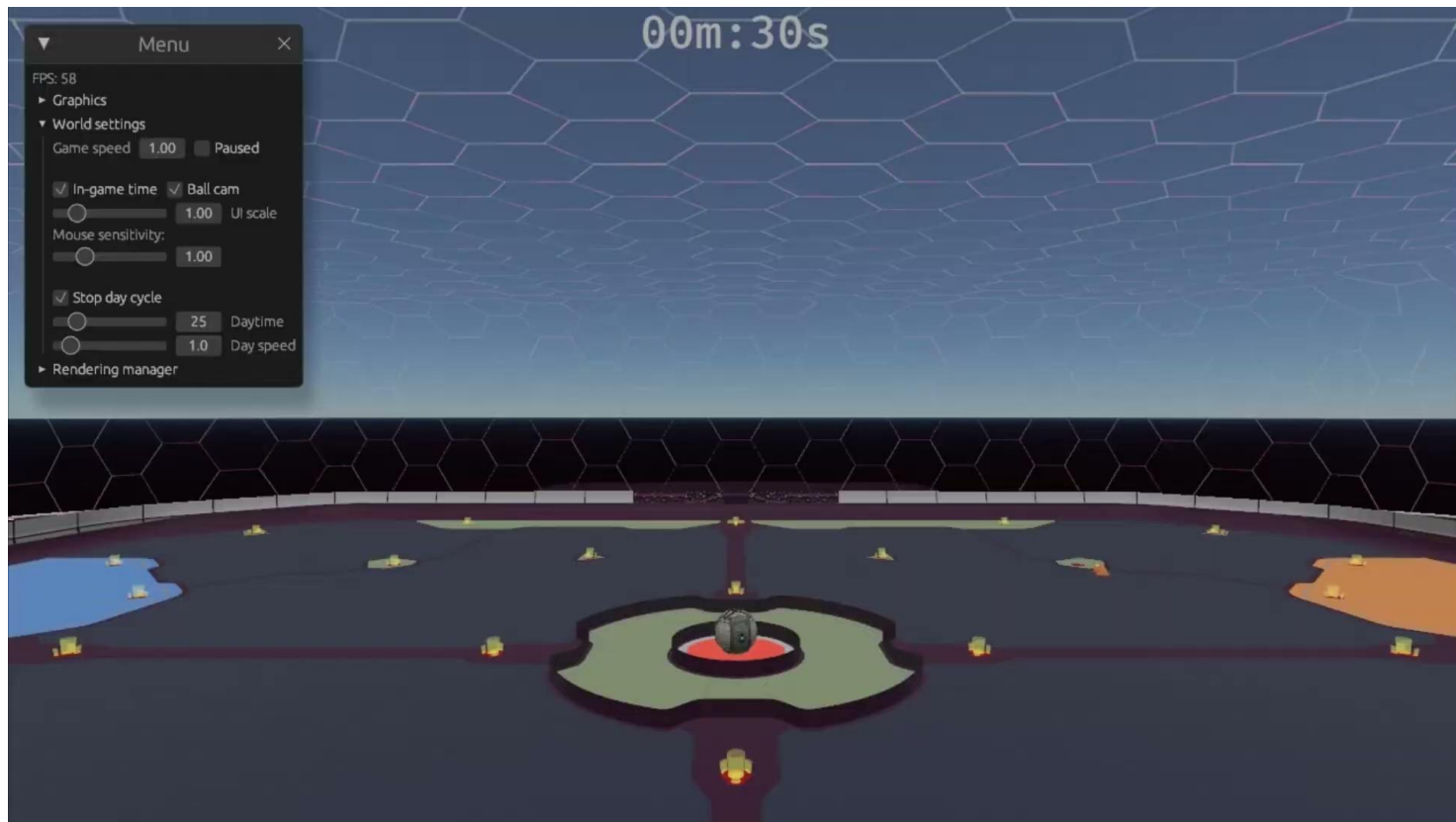
VISUAL DEMO

# DEMO: SPARSE\_40M



<https://drive.google.com/file/d/1eYVI0I2iisNpYHPvyfPsBx8LpehN3P2t/view?usp=sharing>

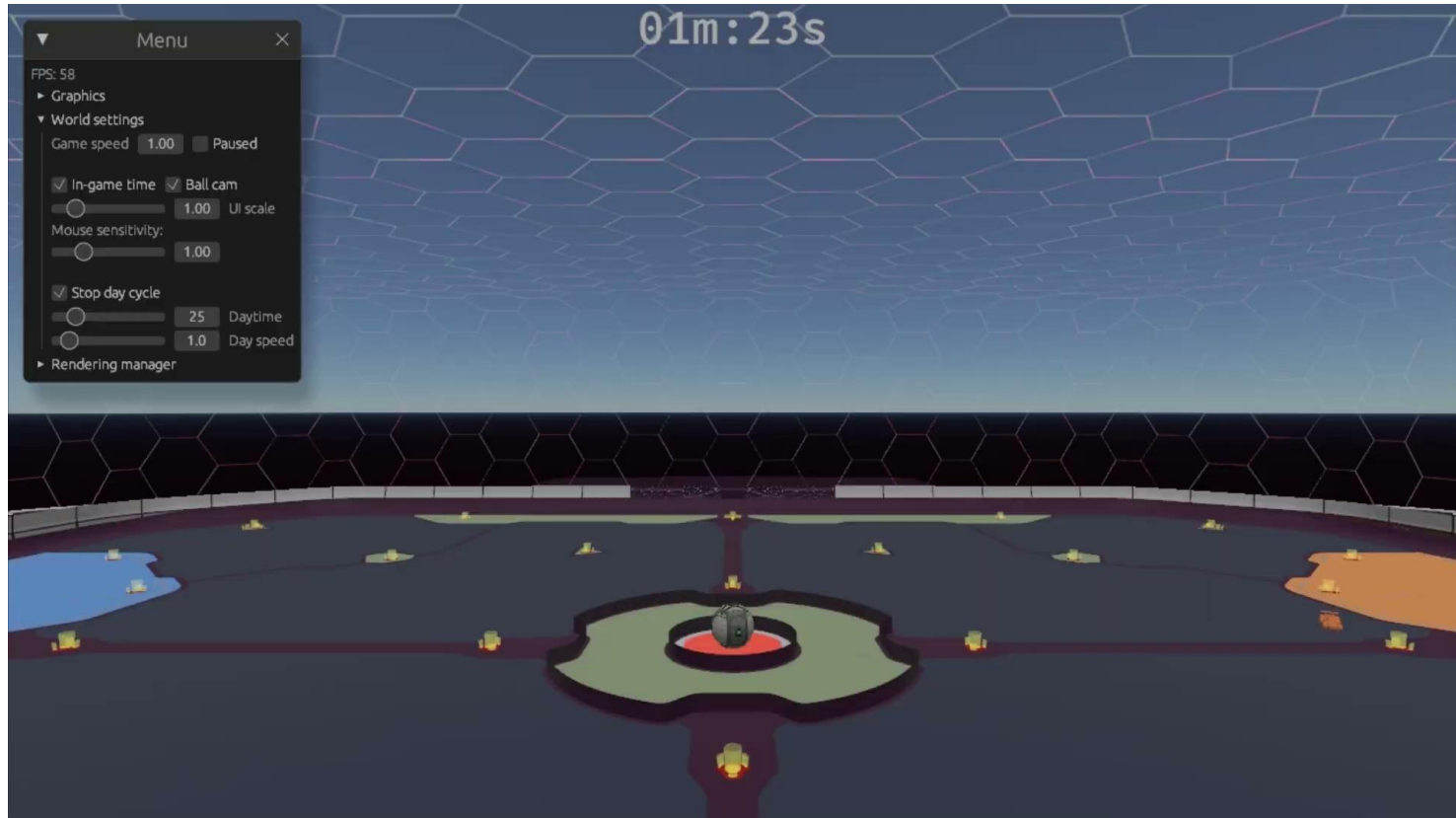
# DEMO: DENSE\_40M



<https://drive.google.com/file/d/1ZQIz9Vw4eBNNCKMKI-vKwDNA-GQ46ITA/view?usp=sharing>

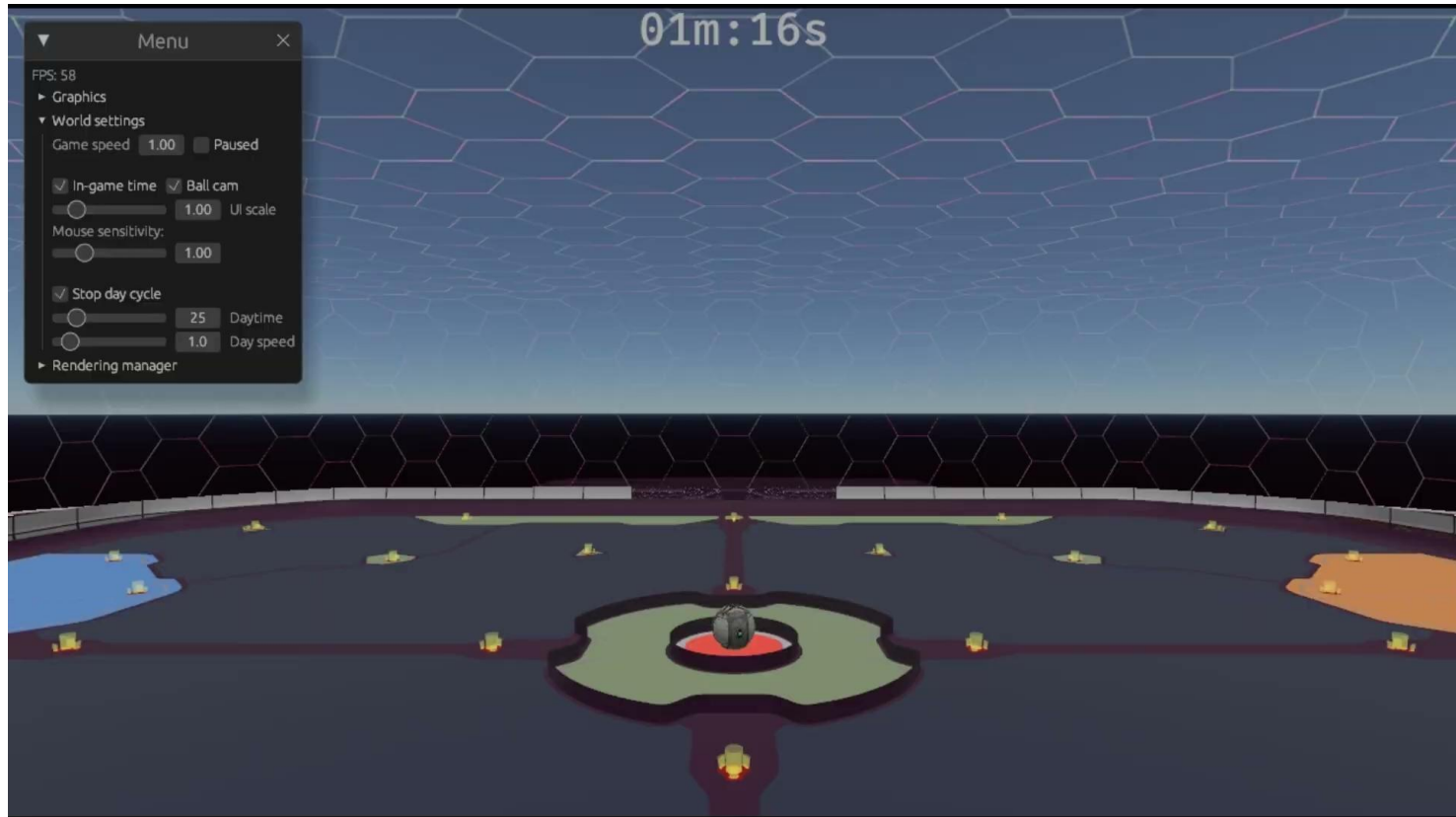


# DEMO: CONTEXT\_40M



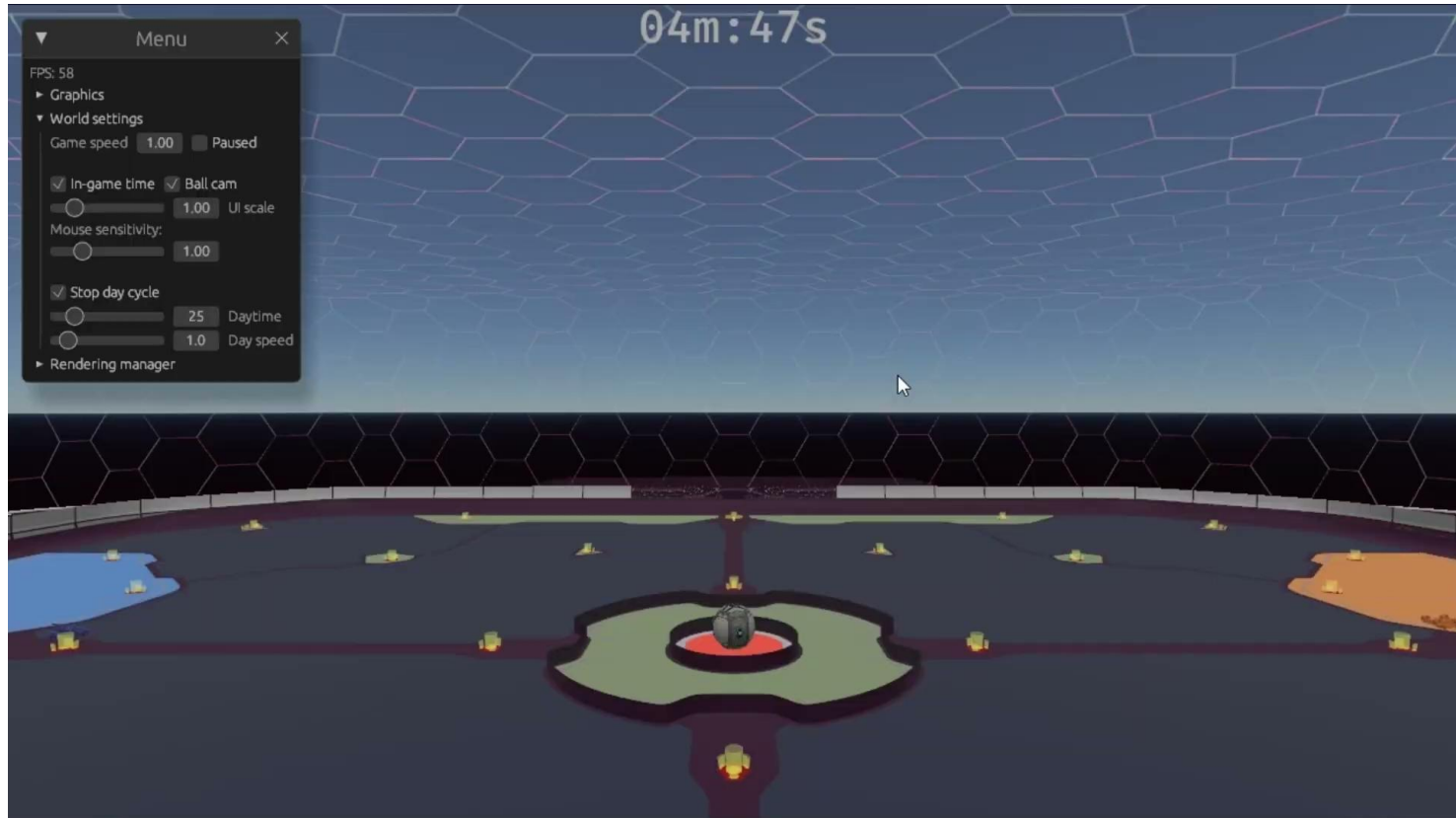
<https://drive.google.com/file/d/15dLHT5llw1QM5xDYnH29SG7xIMWGB0/view?usp=sharing>

# DEMO: CONTEXT\_V2



<https://drive.google.com/file/d/1dCuEvIvFEb1LMDZQiGj6DjJRMKydyFIO/view?usp=sharing>

# DEMO: CONTEXT\_V3



[https://drive.google.com/file/d/1fWNQtuZb-rYYGxIsYWN\\_4OnvjH7IIQZd/view?usp=sharing](https://drive.google.com/file/d/1fWNQtuZb-rYYGxIsYWN_4OnvjH7IIQZd/view?usp=sharing)

Thank You!